

## GeorgeKuttyRenjan\_Shone\_Project\_2\_Report

1. In the PDF report, please report the wall-clock runtime of your parallel program on the four tests using 1, 2, 4, and 8 threads and make a speedup table and an efficiency table. Please comment on the scalability of your parallel code (strongly scalable vs weakly scalable). If you fail to write a correct and scalable algorithm before the deadline, please explain the likely causes in the report and you will receive points for the error analysis.

| Runtime | Test 1   | Test 2   | Test 3    | Test 4    |
|---------|----------|----------|-----------|-----------|
| p = 1   | 4.275740 | 8.845071 | 17.554247 | 46.524366 |
| p = 2   | 2.220734 | 4.430624 | 8.987141  | 23.037758 |
| p = 4   | 1.068443 | 2.276454 | 4.539610  | 12.005718 |
| p = 8   | 0.567607 | 1.134386 | 2.371729  | 5.733502  |

| Speedup | Test 1 | Test 2 | Test 3 | Test 4 |
|---------|--------|--------|--------|--------|
| p = 1   | 1.000  | 1.000  | 1.000  | 1.000  |
| p = 2   | 1.925  | 1.997  | 1.953  | 2.019  |
| p = 4   | 4.002  | 3.884  | 3.868  | 3.873  |
| p = 8   | 7.533  | 7.795  | 7.401  | 8.115  |

| Efficiency | Test 1 | Test 2 | Test 3 | Test 4 |
|------------|--------|--------|--------|--------|
| p = 1      | 1.000  | 1.000  | 1.000  | 1.000  |
| p = 2      | 0.963  | 0.999  | 0.976  | 1.009  |
| p = 4      | 1.000  | 0.971  | 0.967  | 0.968  |
| p = 8      | 0.942  | 0.974  | 0.925  | 1.014  |

The tests are strongly scalable because for each test the runtime decreased proportionally as the thread increased. The speed is also increasing proportionally as the threads increase in each test. The overhead is small because the efficiency stays high.

3. In the report, please describe the algorithm used to solve the problem, including how the data was distributed to the processors. The program should be run using 1, 2, 4, and 8 threads on data of 4 different sizes. For each run, you should time the encryption computation. The report should include 4X4 tables for the computation time, speedup, and efficiency for all these runs. Is your program strongly scalable or weakly scalable and why?

| Runtime | Test 1   | Test 2   | Test 3   | Test 4   |
|---------|----------|----------|----------|----------|
| p = 1   | 0.059475 | 0.118503 | 0.272712 | 0.614695 |
| p = 2   | 0.059293 | 0.118572 | 0.272803 | 0.613436 |
| p = 4   | 0.058906 | 0.118161 | 0.272359 | 0.615987 |
| p = 8   | 0.059371 | 0.118326 | 0.272198 | 0.619505 |

| Speedup | Test 1 | Test 2 | Test 3 | Test 4 |
|---------|--------|--------|--------|--------|
| p = 1   | 1.000  | 1.000  | 1.000  | 1.000  |
| p = 2   | 1.003  | 0.999  | 0.999  | 1.002  |
| p = 4   | 1.010  | 1.003  | 1.001  | 0.998  |
| p = 8   | 1.002  | 1.002  | 1.002  | 0.992  |

| Efficiency | Test 1 | Test 2 | Test 3 | Test 4 |
|------------|--------|--------|--------|--------|
| p = 1      | 1.000  | 1.000  | 1.000  | 1.000  |
| p = 2      | 0.501  | 0.499  | 0.499  | 0.501  |
| p = 4      | 0.253  | 0.251  | 0.250  | 0.249  |
| p = 8      | 0.125  | 0.125  | 0.125  | 0.124  |

The program reads the input text into a byte array of size  $NNN$  and encrypts each byte independently using the formula  $c[i] = (p[i] + k) \bmod 256$ , making the computation embarrassingly parallel. The data is divided evenly among  $p$  threads using block partitioning, where each thread processes a contiguous segment of the array

The tests are strongly scalable because the input size is static while the thread increases.